

ELECTRA: Replaced-Token Detection.

Masked language modelling learns from only the $\sim 15\%$ of positions it hides. ELECTRA asks a cheaper, denser question of *every* token: “was this word in the original sentence, or did a small generator slip it in?” A discriminator answers per token with a sigmoid, and learns from 100% of positions. Every number here — six sigmoid probabilities, six binary cross-entropy terms, and the summed loss — is computed in full, nothing summarised or deferred.

THE EQUATION

$$\mathcal{L}_{\text{disc}} = - \sum_{t=1}^T \left[\mathbb{I}[\text{orig}_t] \log D_t + \mathbb{I}[\text{repl}_t] \log(1 - D_t) \right], \quad D_t = \sigma(w \cdot h_t)$$

Worked example. A 6-token sequence with two generator-replaced tokens; discriminator logits $\rightarrow$ per-token $D_t \rightarrow$ per-token binary cross-entropy $\rightarrow$ total $\mathcal{L}_{\text{disc}}$, plus the sample-efficiency argument.

Topic 2 of 2 in this module · companion to the course page · v1.0

Contents

1	Generator and discriminator	2
2	Worked example: a 6-token sequence	3
3	Sample efficiency: learning from 100% vs. 15%	4
4	Where this sits in the track	5
A	Reproducible (NumPy)	5

1 Generator and discriminator

ELECTRA pretrains *two* networks. A small **generator** G is an ordinary masked language model: mask $\sim 15\%$ of positions and sample a plausible replacement from G 's softmax. The resulting “corrupted” sentence — some tokens original, some swapped for plausible fakes — is fed to a larger **discriminator** D , which classifies *every* token as original or replaced. For token t with final hidden state h_t ,

$$D_t = \sigma(w \cdot h_t) = \frac{1}{1 + e^{-w \cdot h_t}} \quad (1)$$

$D_t \in (0, 1)$ is the probability that token t is *original*. The discriminator loss is the binary cross-entropy over all T positions:

$$\mathcal{L}_{\text{disc}} = - \sum_{t=1}^T \left[\mathbb{I}[\text{orig}_t] \log D_t + \mathbb{I}[\text{repl}_t] \log(1 - D_t) \right] \quad (2)$$

Originals push $D_t \rightarrow 1$; replaced tokens push $D_t \rightarrow 0$. The two networks train jointly with a combined objective,

$$\mathcal{L} = \mathcal{L}_{\text{MLM}}(G) + \lambda \mathcal{L}_{\text{disc}}(D), \quad \lambda \approx 50 \quad (3)$$

The large λ reflects that the discriminator's per-token BCE is numerically much smaller than the generator's MLM cross-entropy, so it must be up-weighted to contribute. Crucially the discriminator gradient is *not* back-propagated through the generator's sampling step; G is trained only by its own MLM loss.

Symbol	Meaning	Shape
h_t	discriminator hidden state at position t	vector in $\mathbb{R}^d$
w	discriminator output weight	vector in $\mathbb{R}^d$
D_t	$P(\text{token } t \text{ is original})$	scalar in $(0, 1)$
$\mathbb{I}[\text{orig}_t]$	1 if token t is original, else 0	$\{0, 1\}$
λ	discriminator-loss weight (≈ 50)	scalar
T	sequence length (all positions scored)	integer

MLM is a teacher who blanks out a few words and asks you to fill them in — you only get graded on those blanks. RTD is a proofreader: it hands you the *whole* sentence, a few words quietly swapped, and asks “flag every word that does not belong.” You are graded on every word, so each sentence teaches far more. And because the fakes come from a competent generator, the fakes are *plausible* — spotting them forces genuine language understanding, not spell-checking.

2 Worked example: a 6-token sequence

Take a sentence of $T = 6$ tokens. The generator replaced tokens at positions 2 and 5 (the rest are original). The discriminator emits one logit $z_t = w \cdot h_t$ per token:

position t	1	2	3	4	5	6
status	orig	repl	orig	orig	repl	orig
logit z_t	2.0	0.5	3.0	1.0	-0.5	2.5

2.1 Step 1 — sigmoid to get D_t (Eq. 1)

$D_t = 1/(1 + e^{-z_t})$. Using $e^{-2} = 0.13534$, $e^{-0.5} = 0.60653$, $e^{-3} = 0.04979$, $e^{-1} = 0.36788$, $e^{0.5} = 1.64872$, $e^{-2.5} = 0.08208$:

$$D_1 = \frac{1}{1+0.13534} = 0.88080, \quad D_2 = \frac{1}{1+0.60653} = 0.62246, \quad D_3 = \frac{1}{1+0.04979} = 0.95257,$$

$$D_4 = \frac{1}{1+0.36788} = 0.73106, \quad D_5 = \frac{1}{1+1.64872} = 0.37754, \quad D_6 = \frac{1}{1+0.08208} = 0.92414.$$

Heatmap of D_t (intensity $\approx D_t \times 80$): high indigo means “confident original”.

	t_1	t_2^r	t_3	t_4	t_5^r	t_6
D_t	0.881	0.622	0.953	0.731	0.378	0.924

The replaced tokens t_2, t_5 are the two lowest cells — the discriminator is already leaning toward “replaced” at t_5 ($D_5 < 0.5$) but is still fooled at t_2 ($D_2 = 0.622$), which will draw a large gradient.

2.2 Step 2 — per-token binary cross-entropy (Eq. 2)

Originals contribute $-\log D_t$; replaced contribute $-\log(1 - D_t)$:

$$\begin{aligned} t_1 (\text{orig}) : -\log(0.88080) &= 0.12693, & t_2 (\text{repl}) : -\log(1 - 0.62246) &= -\log(0.37754) = 0.97408 \\ t_3 (\text{orig}) : -\log(0.95257) &= 0.04859, & t_4 (\text{orig}) : -\log(0.73106) &= 0.31326, \\ t_5 (\text{repl}) : -\log(1 - 0.37754) &= -\log(0.62246) = 0.47408, & t_6 (\text{orig}) : -\log(0.92414) &= 0.07889. \end{aligned}$$

	t_1	t_2^r	t_3	t_4	t_5^r	t_6
BCE_t	0.127	0.974	0.049	0.313	0.474	0.079

(Intensity $\approx \text{BCE} \times 80$, capped at **indigo!80**.) The biggest penalty is at t_2 — the replaced token the discriminator still believes is original.

2.3 Step 3 — sum (Eq. 2)

$$\mathcal{L}_{\text{disc}} = 0.12693 + 0.97408 + 0.04859 + 0.31326 + 0.47408 + 0.07889 = \mathbf{2.01582}.$$

Plugged into the combined objective with $\lambda = 50$, this term contributes $50 \times 2.01582 = 100.79$ alongside the generator’s MLM loss (Eq. 3).

RTD turns pretraining into a per-token binary classification: $D_t = \sigma(w \cdot h_t)$ predicts “original”, scored by binary cross-entropy over *all* T tokens. The loss is dominated by the replaced tokens the model has not yet learned to flag — exactly where the useful gradient is.

3 Sample efficiency: learning from 100% vs. 15%

MLM produces a training signal only at masked positions — about 15% of tokens. RTD produces a signal at *every* position, original and replaced alike. Per sequence, that is

$$\frac{100\%}{15\%} \approx 6.7\times$$

more supervised positions. This is the core reason ELECTRA matches BERT/RoBERTa accuracy with far less compute: it extracts roughly $6\text{--}7\times$ more signal from each sequence, so it reaches the same quality in a small fraction of the pretraining FLOPs (a key win for a budget-constrained pretrain).

1. **Dense signal.** Loss at T positions, not $0.15T$.
2. **No [MASK] mismatch.** The discriminator never sees an artificial [MASK] token, so there is no pretrain/fine-tune input distribution gap (BERT sees [MASK] in pretraining but never at fine-tuning).
3. **Small generator.** G is intentionally small (often $\frac{1}{4}\text{--}\frac{1}{2}$ of D); a *too-strong* generator makes detection too hard and hurts, so it is sized down, keeping its cost low.

Keeping the generator at inference time. After pretraining the generator G is **discarded** — only the discriminator D is fine-tuned on downstream tasks. Shipping or fine-tuning the generator by mistake gives you a weak MLM, not ELECTRA. Also note RTD is *binary* (sigmoid + BCE per token), not a softmax over the vocabulary: scoring it as multi-class cross-entropy over tokens is a different, wrong objective.

4 Where this sits in the track

ELECTRA closes the encoder-pretraining arc. Module 1.1 established BERT’s MLM (and its 80/10/10 masking and combined loss); Topic 1 of this module sharpened the *architecture* with DeBERTa’s disentangled attention. ELECTRA instead keeps a conventional Transformer encoder but swaps the *objective* for replaced-token detection — denser supervision, no [MASK] gap, far less compute — yielding a strong encoder for the same downstream uses (classification, retrieval, sequence labelling) that the rest of Phase 1 targets.

A Reproducible (NumPy)

Inputs: status vector (1 = original, 0 = replaced) and discriminator logits as above. The snippet regenerates every D_t , the per-token BCE, and the total $\mathcal{L}_{\text{disc}}$.

```
import numpy as np
orig = np.array([1,0,1,1,0,1])          # 1=original, 0=replaced (t2,t5)
logits = np.array([2.0,0.5,3.0,1.0,-0.5,2.5]) # w . h_t per token
D = 1/(1+np.exp(-logits))                # sigmoid -> P(original)
bce = np.where(orig==1, -np.log(D), -np.log(1-D))
print(np.round(D,5))                     # [0.8808  0.62246 0.95257 0.73106 0.37754 0.92414]
print(np.round(bce,5))                   # [0.12693 0.97408 0.04859 0.31326 0.47408 0.07889]
print(bce.sum())                         # 2.0158207527877003
print(100/15)                           # 6.666... (signal-density ratio, MLM 15% -> RTD 100%)
```